

# Modelling Software-based Systems

## Lecture 4

### Validation, Verification and Proof Tools for Event-B for Correct-by-Construction

Telecom Nancy (IL et LE)

Dominique Méry  
Telecom Nancy, Université de Lorraine

30 septembre 2026  
dominique.mery@loria.fr

- 1 Proof Management
- 2 Development Methodology and Tools for Correct By Construction
- 3 Project Management

## Current Summary

- 1 Proof Management
- 2 Development Methodology and Tools for Correct By Construction
- 3 Project Management

- The panel called *proof control* is providing tools for *discharging* proof obligations
- Proof obligations are stated as follows :  $\Gamma \vdash P$  : *When assertions of  $\Gamma$  are supposed to hold, then  $P$  does hold*
- Problem : Checking that the sequent  $\Gamma \vdash P$  is valid.
- How : Applying automatic procedures as pp, pr etc
- ... or helping the proof tool to build the *proof tree*

- Quite sensitive to useless hypotheses.
- Unable to automatically abstract some statements.
- Sometimes poor management of equalities.
- Needs some creative hints



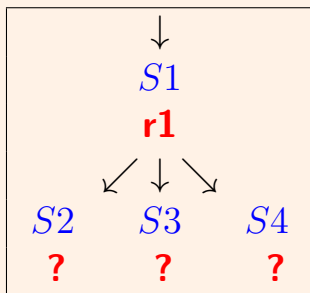
- **Sequent** (where *list\_of\_hypotheses* might be empty).

$$\textit{list\_of\_hypotheses} \vdash \textit{conclusion}$$

- **Inference rule** (where *list\_of\_sequents* might be empty).

$$\frac{\textit{list\_of\_sequents}}{\textit{sequent}}$$

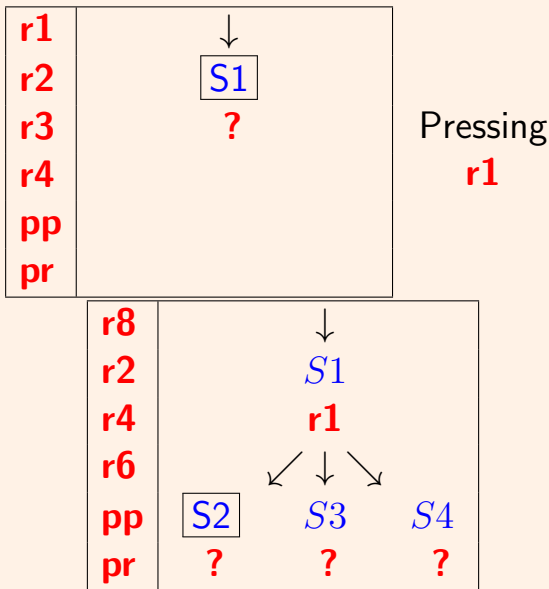
- **Backwards application** of inf. rules  $\leadsto$  **proof tree**.



- Rule  $r1$  is applied to sequent  $S1$ .
- Producing sequents  $S2$ ,  $S3$ , and  $S4$ .

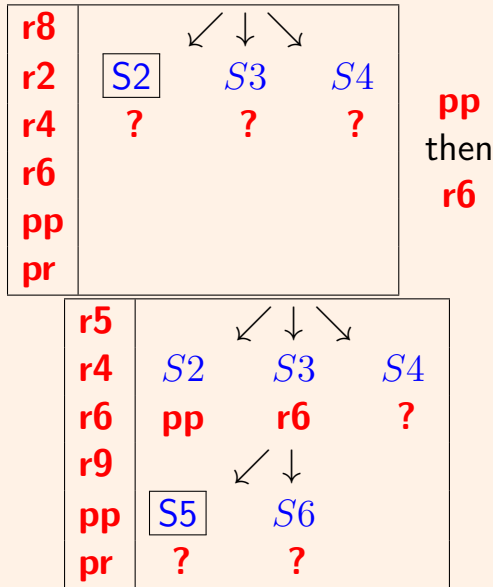


## Interface : Tree and Palette of Rule Buttons



- The proof tree is **constructed interactively**.

## Difficulty : Size of Window



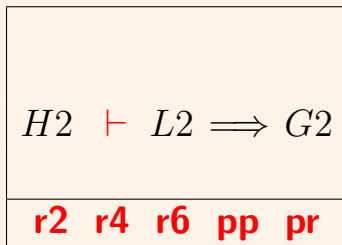
- Our tentative “interface” does not work.

$$H \vdash L \Longrightarrow G$$

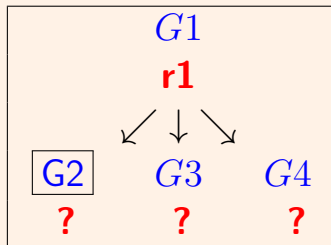
- Most of the time the conclusion has an **implicative form**
- $G$  is called the **goal** (it is a **non-conjunctive** predicate)
- $L$  are called the **local hypotheses** (might be empty)

$$H \vdash I(x) \wedge G(x) \wedge P(x, x') \Longrightarrow I(x')$$

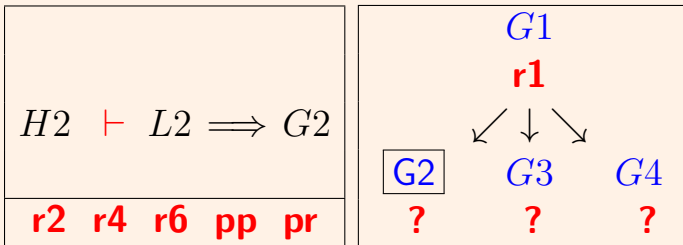
- **Tree window** with **simplified** sequents (**goals** only)
- **Sequent window** containing the **full sequent of interest**



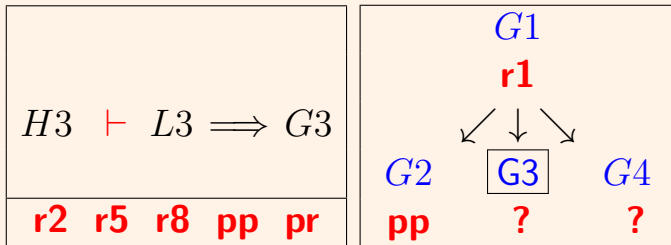
Sequent Window

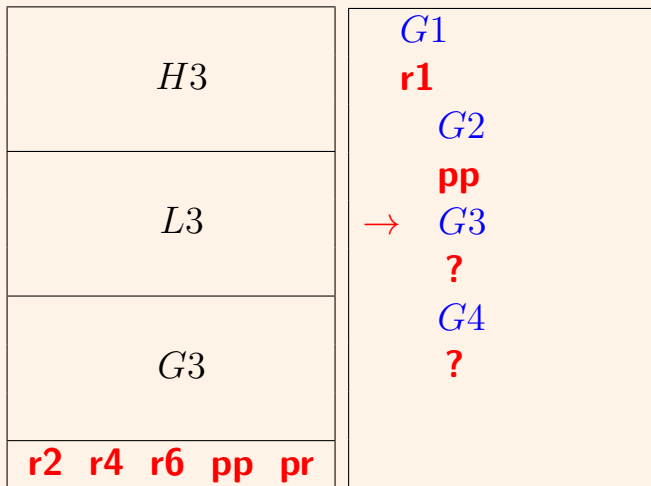


Tree Window



Pressing **pp**





$$\underbrace{hidden ; searched ; cached}_{list\_of\_hypotheses} \vdash \underbrace{local \implies goal}_{conclusion}$$

|            |                                                     |
|------------|-----------------------------------------------------|
| $hidden$   | <b>Not visible</b> on the sequent window            |
| $searched$ | <b>Visible</b> after some search in $hidden$        |
| $cached$   | <b>Visible</b> but not part of the conclusion any m |
| $local$    | <b>Visible</b> and part of the conclusion           |

## Current Summary

- 1 Proof Management
- 2 Development Methodology and Tools for Correct By Construction
- 3 Project Management



## Context

Developing life-cycle methodology and a set of associated techniques and tools to develop the highly critical systems using formal techniques from requirements analysis to code generation.

# Objectives

- To establish a unified theory for the critical system development.
- To Build a set of tools for supporting new development life-cycle methodology.
- To develop a closed-loop system for verification purpose.
- Graphical based refinement technique to handle the complexity of the system.
- To satisfy requirements and metrics for certifiable assurance and safety.
- To support evidence-based certification.

## Development Methodology

- Formal Methods based Critical System Development Life-Cycle

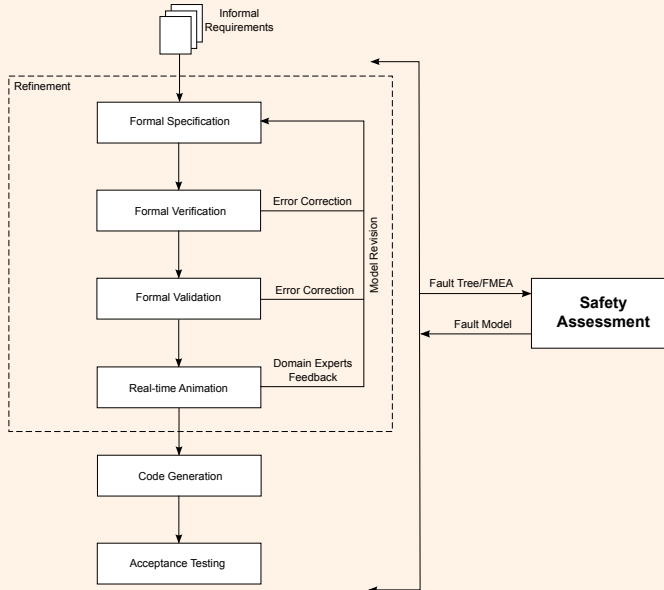
## Techniques and Tools

- Real-Time Animator
- Refinement Chart
- Automatic Code Generator : [EB2ALL](#)
- Formal Logic Based Heart-Model

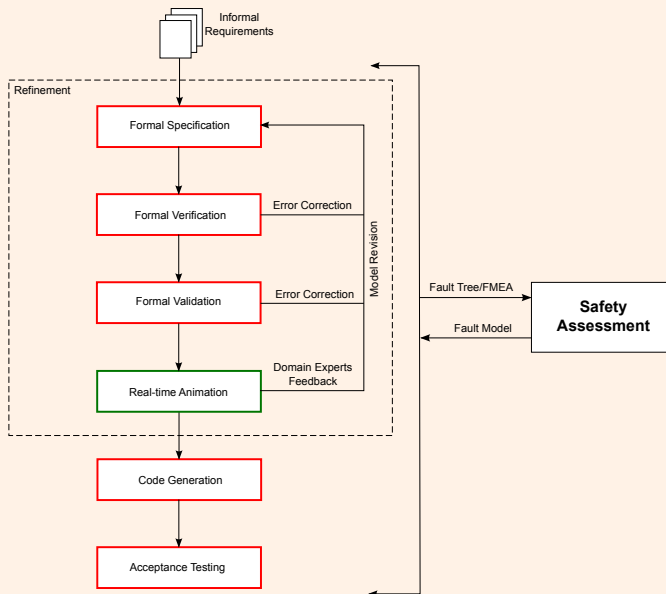
## Case Studies

- The Cardiac Pacemaker

# Critical System Development Life-Cycle Methodology



# Critical System Development Life-Cycle Methodology



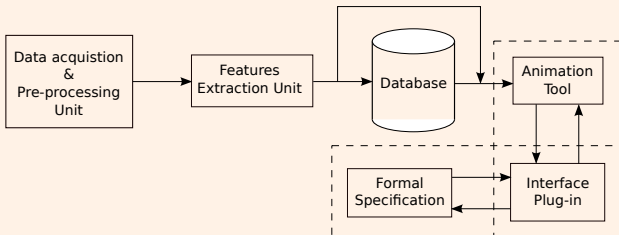
## What is Real-time Animator ?

- Visual representation of formal model using real time data set.

## Why should we use Formal Model Animator ?

- To validate system behavior according to the stakeholders
- To express formal models for non-mathematical domain experts
- To discover the error in the early stage of system development (Traceability)

## Proposed Architecture

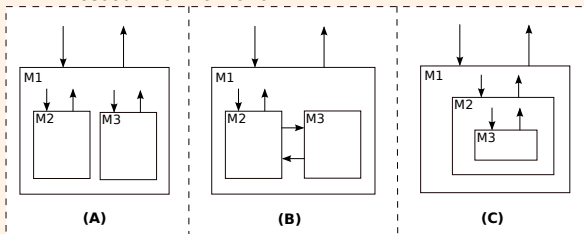


- Refinement chart is a graphical representation of a complex system using layering approach.
- Integration and features based code structuring of different components of the systems.

## Basic rules of refinement chart

- Parallel Refinement
- Sequential Refinement
- Nested Refinement

$$\begin{aligned} M1 &\sqsubseteq (M2 \parallel M3 \parallel \dots \parallel M_{n-1} \parallel M_n) \\ M1 &\sqsubseteq (M2 \succ M3 \succ \dots \succ M_{n-1} \succ M_n) \\ (M1 &\sqsubseteq M2, M2 \sqsubseteq M3, \dots, M_{n-1} \sqsubseteq M_n) \end{aligned}$$



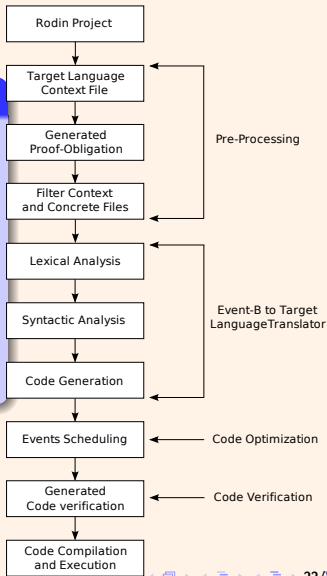
$$\begin{aligned} M1 &\sqsubseteq (M2 \parallel M3) \\ M1 &\sqsubseteq (M2 \succ M3) \\ (M1 &\sqsubseteq M2, M2 \sqsubseteq M3) \end{aligned}$$

## EB2ALL

- Automatic code generation from Event-B formal specification to multiple languages (C, C++, Java, C#) to make it executable.
- A module for Rodin (Deploy is an European Commission FP7 project)
- It is more than 20000 lines of code
- More than 1000 downloads

Download : <http://eb2all.loria.fr/>

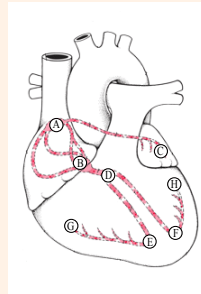
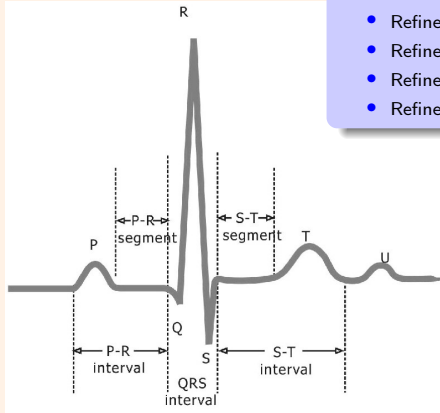
## EB2ALL Tool Architecture



## Formal Development

- Abstract Model
- Refinement 1 : Introducing Steps in the Propagation
- Refinement 2 : Impulse Propagation
- Refinement 3 : Perturbation the Conduction
- Refinement 4 : Getting a Cellular Model

Electrocardiogram (ECG)



Landmarks on the Electrical Conduction of Heart



## Current Summary

- 1 Proof Management
- 2 Development Methodology and Tools for Correct By Construction
- 3 Project Management

- Reading the description of the problem to solve.
- List the safety properties as partial correctness, mutual exclusion, ... and properties on sets, constants, ...
- Identify the state variables for the first machine
- Define inductive definitions if necessary and state correctness properties as  $\forall n. n \in \mathbb{N} \Rightarrow n * n = v(n)$ .
- List Use Cases : for instance, adding a new user, updating a a phone number, recording a movement ...
- First expression of a context and a machine.

